

Toy Story: Beat the Beetles

Tiziana Mascolo

Simone Martucci

Esame di

Metodi Avanzati di Programmazione

(track M-Z)

A.A. 2025/2026

Descrizione dell'avventura

«Toy Story: Beat the Beetles» è un'avventura grafica punta-e-clicca cooperativa e multiplayer, ispirata all'universo di Toy Story. I giocatori controllano a turno Woody, Buzz Lightyear e Jessie, in una casa invasa da un'orda di scarafaggi che minaccia la torta di compleanno di Andy. Obiettivo dell'avventura è esplorare le stanze della casa e delle fogne sottostanti, raccogliere oggetti, risolvere enigmi ambientali e raggiungere lo scontro finale con il «boss», sfruttando le abilità uniche di ciascun personaggio (il laser di Buzz, la destrezza di Jessie, il lazo che Woody sblocca nel corso della trama).

Il gioco è multiplayer: più client possono collegarsi alla stessa partita (identificata da un codice a 6 caratteri) e ciascuno può assumere il controllo di uno dei personaggi liberi, passando da uno all'altro con il comando «CHIAMA»; il server impedisce che due giocatori controllino lo stesso personaggio contemporaneamente. Le azioni disponibili sono GUARDA, PRENDI, USA, VAI, PARLA e CHIAMA, selezionabili dalla pulsantiera e applicate cliccando sulle aree interattive dello scenario. Lo stato della partita (stanze, oggetti, inventari, flag di progressione della trama) è persistente su database, così da poter sospendere e riprendere una partita in qualsiasi momento tramite il codice partita.

Progettazione

Le classi sono state individuate analizzando il dominio dell'avventura (stanze, oggetti, personaggi, comandi) e tenendolo separato dall'infrastruttura tecnica (rete, persistenza, interfaccia grafica).

Il progetto è diviso in due moduli indipendenti — client e server — che comunicano esclusivamente tramite socket TCP con un protocollo testuale a campi separati da «|»: il server è l'unico detentore dello stato di gioco, il client si limita a presentare l'interfaccia e a inoltrare i comandi dell'utente:

- **com.toystory.server** → infrastruttura di rete e orchestrazione: ServerMain, ServerThread, GameSession, ClientState, Engine, GameObservable/GameObserver, GameDescription.
- **com.toystory.server.type** → modello di dominio riutilizzabile e indipendente dalla specifica avventura: Room, AdvObject, GameCharacter, PickupableObject, PlayableCharacter, Ability, Command, CommandType, ActionResult.
- **com.toystory.server.impl** → implementazione concreta di questa avventura: ToyStoryGame (estende GameDescription e costruisce le 16 stanze, i personaggi e i flag di trama), i sei Observer di comando (LookAtObserver, PickupObserver, UseObserver, MoveObserver, CallObserver, TalkObserver) e Dialoghi, che centralizza i testi narrativi.
- **com.toystory.server.database** → DatabaseManager, unico responsabile dell'accesso JDBC/H2 e delle transazioni.
- **com.toystory.client** (con i sotto-package .view e .view.components) → interfaccia grafica Swing organizzata secondo il pattern MVC: GameWindow (view), GameWindowController (controller), GUIHandler (interprete del protocollo di rete verso chiamate grafiche), MappaScenario/BoundingBox (modello delle aree cliccabili), GameClient (livello di rete), GameDialogs, PannelloImmagineAdattiva e PixelButtonGenerator (componenti grafici riutilizzabili).

Ogni classe ha quindi una competenza singola e ben definita:

ServerThread si occupa solo di I/O di rete verso un client,

GameSession solo di coordinare i giocatori di una singola partita,

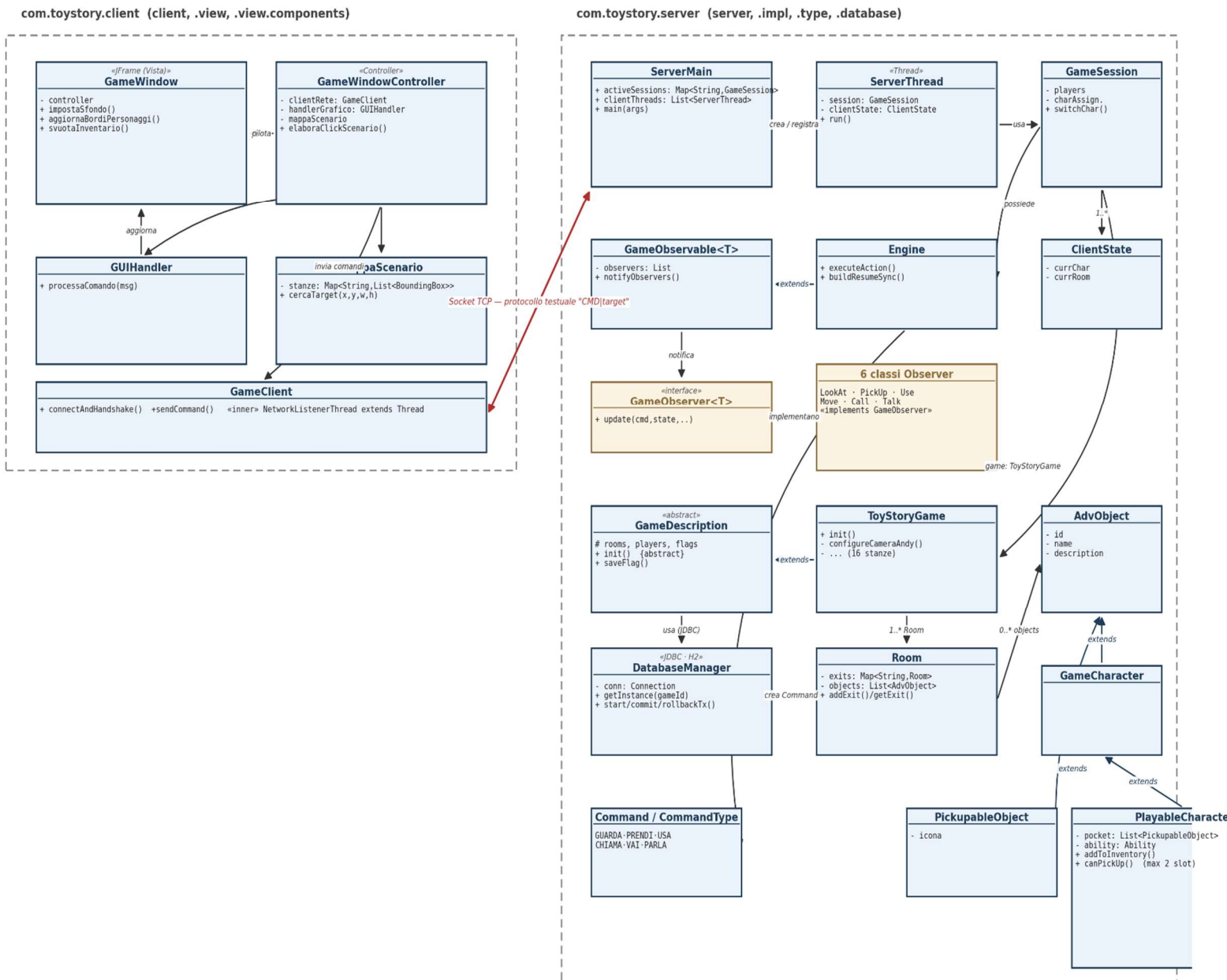
Engine solo di smistare i comandi agli Observer,

DatabaseManager solo di persistenza.

Questa separazione permette, ad esempio, di aggiungere un nuovo comando di gioco implementando un nuovo *GameObserver* senza modificare il resto del sistema.

Diagramma delle classi

Il diagramma seguente mostra l'architettura completa client-server del progetto: il livello di rete (socket TCP), il pattern Observer che implementa la logica dei comandi lato server e la gerarchia del modello di dominio (stanze, personaggi, oggetti).



Principi della programmazione ad oggetti utilizzati:

- **Ereditarietà:**

AdvObject → GameCharacter → PlayableCharacter e AdvObject → PickupableObject formano la gerarchia degli oggetti di gioco; GameDescription (astratta) → ToyStoryGame separa la parte generica di un motore di avventura da quella specifica di questa storia.

- **Interfacce:**

GameObserver<T> definisce il contratto update(Command, GameDescription, ClientState, GameSession) implementato in modo indipendente dalle sei classi di comando, permettendo di aggiungerne di nuove senza modificare Engine.

- **Classi astratte:**

GameDescription fissa la struttura comune di un'avventura (stanze, personaggi, flag, persistenza) lasciando a ToyStoryGame il solo compito di popolare il mondo di gioco tramite init().

- **Composizione:**

GameSession è composta da un Engine, un DatabaseManager e una mappa di ClientState (uno per giocatore); Room aggrega una lista di AdvObject e una mappa di uscite verso altre Room; PlayableCharacter aggrega la propria «tasca» di PickupableObject.

- **Polimorfismo:**

Engine.notifyObservers() invoca lo stesso metodo update() su oggetti di tipo dinamico diverso (i sei Observer), senza conoscerne la classe concreta — solo l'Observer competente per il comando ricevuto restituisce una risposta non nulla.

- **Generics:**

GameObserver<T> e GameObservable<T> sono parametrizzati sul tipo di risposta (String nel protocollo attuale, ma riutilizzabili con altri tipi); ActionResult<T> incapsula in modo generico l'esito booleano, il messaggio e un payload opzionale di un'azione.

Specifica algebrica

struttura lista: la tasca (pocket) di PlayableCharacter, una lista di PickupableObject con capienza massima di 2 oggetti.

Specifica sintattica

sorts: pocket, item, integer, boolean

operations:

- newPocket() -> pocket
- add(pocket, item) -> pocket
- remove(pocket, item) -> pocket
- size(pocket) -> integer
- isFull(pocket) -> boolean
- contains(pocket, item) -> boolean

Costruttori e osservazioni

Per il criterio di minimalità, bastano newPocket() e add per costruire qualunque tasca possibile: sono quindi i costruttori. remove, pur restituendo un pocket, è un'osservazione, perché non serve a costruire nuovi valori ma solo a «disfare» ciò che add ha costruito.

	newPocket()	add(p, i)
size(p')	0	size(p) + 1
isFull(p')	false	size(p) + 1 = 2
contains(p', i')	false	if i=i' then true else contains(p, i')
remove(p', i')	error	if i=i' then p else add(remove(p, i'), i)

Specifica semantica

declare p: pocket, i, i': item

- $\text{size}(\text{newPocket}()) = 0$
- $\text{size}(\text{add}(p, i)) = \text{size}(p) + 1$
- $\text{isFull}(\text{newPocket}()) = \text{false}$
- $\text{isFull}(\text{add}(p, i)) = (\text{size}(p) + 1 = 2)$
- $\text{contains}(\text{newPocket}(), i') = \text{false}$
- $\text{contains}(\text{add}(p, i), i') = \text{if } i = i' \text{ then true else contains}(p, i')$
- $\text{remove}(\text{add}(p, i), i') = \text{if } i = i' \text{ then } p \text{ else add}(\text{remove}(p, i'), i)$

Queste equazioni sono sufficienti e nessuna è ridondante: ognuna riconduce l'applicazione di un'osservazione su $\text{add}(p, i)$ al caso più piccolo p , esattamente come $\text{pop}(\text{push}(\text{stk}, i)) = \text{stk}$ riconduce la Pila al caso base.

Specifica di restrizione

restrictions

- $\text{add}(p, i) = \text{error, se } \text{isFull}(p) = \text{true}$
- $\text{remove}(\text{newPocket}(), i') = \text{error}$

La prima restrizione è l'aspetto più caratterizzante, la tasca ha una capienza fissa di 2 oggetti, quindi add è un'operazione parziale, indefinita quando la tasca è già piena.

Dettagli implementativi

Per ciascun argomento del corso spiegare se e come è stato utilizzato all'interno del progetto.

Programmazione generica

Il progetto usa i generics in tre punti chiave. L'interfaccia `GameObserver<T>` e la classe `GameObservable<T>` sono parametrizzate sul tipo di risposta prodotta da un'azione di gioco (attualmente `String`, il protocollo testuale verso il client): questo disaccoppia il meccanismo di notifica degli osservatori dal tipo concreto restituito. `ActionResult<T>` è un wrapper generico per l'esito di un'operazione (successo/fallimento, messaggio, payload opzionale di tipo `T`): in `PickUpObserver` viene istanziato come `ActionResult<PickupableObject>`, così la stessa classe può rappresentare l'esito di qualsiasi azione senza duplicare codice per ogni tipo di payload. Infine `DatabaseManager.getFlagAs(String key, Class<T> type)` è un metodo generico che, grazie al token di tipo `Class<T>`, converte a runtime il valore testuale letto dal database nel tipo richiesto (`Boolean`, `Integer` o `String`), aggirando la cancellazione dei generics di Java.

File

Tutti i testi narrativi del gioco (dialoghi) sono externalizzati nel file di testo `config/dialoghi.txt`, nel formato `CHIAVE=testo`, evitando un'implementazione monolitica nel codice sorgente. La classe `Dialoghi` li carica una sola volta, in un blocco static, tramite `File` (per verificarne l'esistenza prima dell'apertura) e `BufferedReader/FileReader` con charset UTF-8 esplicito, per gestire correttamente gli accenti; la lettura è riga per riga, ignorando righe vuote e commenti (che iniziano con `#`), e i valori vengono memorizzati in una `HashMap<String,String>`. Lo stream viene sempre chiuso in un blocco `finally`, anche in presenza di eccezioni. Questa scelta separa nettamente il contenuto narrativo dalla logica di gioco, permettendo di correggere o tradurre i testi senza ricompilare il codice.

Database (JDBC)

La persistenza è realizzata con JDBC su database H2 embedded. `DatabaseManager` è un factory singleton per partita: `getInstance(gameId)` restituisce, tramite una `ConcurrentHashMap`, un'unica connessione per ogni gameId, con un file fisico separato (`saves/toystory_<ID>.mv.db`) così da poter ospitare più partite contemporaneamente sullo stesso server. Lo schema è composto da quattro tabelle: `game_flags` (flag di progressione della trama), `rooms`, `game_objects` (posizione degli oggetti) e `inventory` (oggetti in tasca ai personaggi).

Tutte le query usano `PreparedStatement` per evitare SQL injection; il salvataggio dei flag sfrutta l'istruzione `MERGE INTO` di H2 per fare un upsert atomico. Ogni azione del giocatore viene incapsulata in una transazione esplicita

(`startTransaction/commitTransaction/rollbackTransaction`), così che un errore durante l'esecuzione di un'azione non lasci lo stato del database inconsistente.

All'avvio, se il database è vuoto `GameDescription` lo popola con la mappa iniziale; altrimenti ricarica da esso lo stato salvato (stanze, posizione degli oggetti, inventari, flag), permettendo di riprendere una partita interrotta.

Lambda Expression (compresi stream e pipeline)

Le lambda expression e la Stream API sono usate diffusamente lato server per interrogare e filtrare le collezioni del modello di dominio, e lato client per disaccoppiare la rete dalla GUI.

Alcuni esempi significativi:

- in [UseObserver/PickUpObserver](#), per verificare se un personaggio possiede un determinato oggetto.

```
attivo.getPocket().stream().anyMatch(obj ->
obj.getName().equalsIgnoreCase("chiave"))
```
- in [PickUpObserver.tentaRaccolta\(\)](#), per cercare un oggetto nella stanza corrente.

```
currentRoom.getObjects().stream().filter(obj ->
obj.getName().equalsIgnoreCase(nome)).findFirst().orElse(null)
```
- in [GameSession.releaseClient\(\)](#), per liberare tutti i personaggi controllati da un client disconnesso.

```
characterAssignments.entrySet().removeIf(e -> e.getValue() == client)
```
- per creare in modo atomico una [GameSession](#) solo se non esiste già, evitando race condition tra due client che si uniscono nello stesso istante.

```
ServerMain.activeSessions.computeIfAbsent(gameId, id -> new GameSession(id))
```
- in [GameWindowController](#), passato come lambda a GameClient: disaccoppia completamente il livello di rete dalla logica di aggiornamento della GUI, secondo un pattern equivalente all'Observer ma basato su interfaccia funzionale.

```
new Consumer<String>(messaggio -> SwingUtilities.invokeLater(() ->
handlerGrafico.processaComando(...)))
```
- in [ServerMain](#), per garantire la chiusura ordinata di tutte le connessioni al database allo spegnimento del server.

```
Runtime.getRuntime().addShutdownHook(new Thread(() -> { /* chiude tutti i DB */
}))
```

SWING

L'interfaccia grafica è realizzata interamente in Swing ed estesa con logica personalizzata. Il progetto rispetta rigorosamente il pattern MVC: GameWindow espone solo metodi pubblici di aggiornamento della vista (impostaSfondo(), aggiungiAllInventario(), aggiornaBordiPersonaggi(), ecc.). Sono stati realizzati componenti grafici custom: PannelloImmagineAdattiva estende JPanel e ridefinisce paintComponent() per disegnare lo sfondo della stanza scalandolo dinamicamente alle dimensioni del pannello; PixelButtonGenerator genera a runtime, con Graphics2D su BufferedImage, sia le icone dei pulsanti verbo in stile «pixel art», sia bordi custom (Border) disegnati proceduralmente (es. la cornice a lavagna magica o la cassa di legno per l'inventario). GameDialogs centralizza tutte le finestre modali (JOptionPane) per garantire coerenza visiva.

Thread e programmazione concorrente

La concorrenza è usata in modo estensivo sia lato server sia lato client.

Sul server, ServerMain accetta connessioni in un ciclo bloccante e avvia un ServerThread (estende Thread) dedicato per ciascun client connesso, così da poter gestire più giocatori simultaneamente senza che la lettura bloccante sulla socket di uno blocchi gli altri.

Le strutture condivise fra i thread sono protette: clientThreads è una List sincronizzata, activeSessions è una ConcurrentHashMap (con computeIfAbsent atomico per evitare la creazione doppia di una sessione).

All'interno di una singola partita, le azioni dei giocatori sono serializzate con un blocco synchronized(sessionEngine) in ServerThread e con metodi synchronized in GameSession (assignInitialCharacter, switchCharacter, releaseClient, broadcast): questo impedisce corse critiche sullo stato di gioco e sul database quando più giocatori della stessa partita agiscono nello stesso istante, mentre partite diverse restano completamente indipendenti e parallele fra loro.

Anche le operazioni transazionali di DatabaseManager (startTransaction/commitTransaction/rollbackTransaction) sono synchronized.

Sul client, GameClient avvia un thread interno dedicato, NetworkListenerThread, che resta bloccato in lettura sulla socket senza congelare l'interfaccia grafica; una variabile volatile running garantisce che il segnale di chiusura sia visibile in modo sicuro fra il thread di rete e quello che richiede la disconnessione.

Socket e/o REST

La comunicazione client-server non usa REST/HTTP ma socket TCP grezze (java.net.Socket / ServerSocket), sulla porta 6666. ServerMain esegue il tradizionale ciclo accept() e delega ogni connessione a un ServerThread dedicato.

Il protocollo applicativo è testuale e a righe: ogni messaggio termina con newline e i campi sono separati dal carattere «|» (es. il client invia «PRENDI|chiave», il server può rispondere con più token concatenati sulla stessa riga come «TESTO|Hai preso la chiave|INVENTARIO|chiave|chiave.png»).

Lato client, GUIHandler.processaComando() esegue il parsing con un cursore che avanza di un numero di token variabile a seconda del comando riconosciuto.

La connessione prevede una fase di handshake iniziale (CREA_PARTITA per creare una nuova partita, UNISCITI_PARTITA|<id> per unirsi a una esistente) prima di entrare nel ciclo di gioco vero e proprio, in cui il server resta in ascolto dei comandi e risponde in modo sincro al mittente.